CO3 AT3-ANALYTICAL PROBLEM SOLVING

SUTDENT NAME: P. VISHNU VARDHAN REDDY

REG. NO: 192425212

COURSE CODE: DSA0108

Question 1: Banking Account Management System

Code:

```
#include <iostream>
#include <string>
using namespace std;

class Account {
private:
int accountNumber;
string customerName;
double balance;

public:
// 1. Default constructor
Account() {
accountNumber = 0;
customerName = "Unknown";
balance = 0.0;
cout << "Default Constructor called." << endl;
}

// 2. Parameterized constructor
Account(int accNo, string name, double bal) {
accountNumber = accNo;
customerName = name;
balance = bal;
cout << "Parameterized Constructor called." << endl;
```

```
}

// 3. Copy constructor
Account(const Account &other) {
accountNumber = other.accountNumber;
customerName = other.customerName;
balance = other.balance;
cout << "Copy Constructor called." << endl;
}

// Function to display account details
void display() {
cout << "Account Number : " << accountNumber << endl;
cout << "Customer Name : " << customerName << endl;
cout << "Balance : " << balance << endl;
cout << "-----------------------------" << endl;
}
};

int main() {
cout << "Creating Account using Default Constructor:" << endl;
Account acc1; // Default constructor
acc1.display();

cout << "\nCreating Account using Parameterized Constructor:" <<
endl;
Account acc2(101, "Vishnu Vardhan", 5000.75); // Parameterized
constructor
acc2.display();

cout << "\nCreating Account using Copy Constructor:" << endl;
Account acc3(acc2); // Copy constructor
acc3.display();

return 0;
}
```

Output:

```
Creating Account using Default Constructor:
Default Constructor called.
Account Number : 0
Customer Name : Unknown
Balance : 0
-----------------------------
```

```
Creating Account using Parameterized Constructor:
Parameterized Constructor called.
Account Number : 101
Customer Name : Vishnu Vardhan
Balance : 5000.75
-----------------------------
```

```
Creating Account using Copy Constructor:
Copy Constructor called.
Account Number : 101
Customer Name : Vishnu Vardhan
Balance : 5000.75
-----------------------------
```

Task 5: How object initialization differs for the three constructors

The three constructors initialize objects differently based on the availability of data at the time of object creation:

- The **Default constructor** takes no arguments and assigns fixed default values (accountNumber = 0, customerName = "Unknown", balance = 0.0) to the data members. It is used when no specific information is available at the time of object creation.
- The **Parameterized constructor** takes arguments (account number, name, and balance) from the caller and directly assigns these passed values to the data members, allowing custom initialization for each object.
- The **Copy constructor** takes a reference to an existing Account object as its parameter and copies each data member's value from that object into the new object, effectively duplicating an already existing account rather than creating one from scratch.

Task 7: Order in which constructors are called during object creation and copying

When the program runs, the constructors are called in the following order:

1. First, the Default Constructor is called when "acc1" is declared with no arguments.

2. Next, the Parameterized Constructor is called when "acc2" is declared with three arguments (account number, name, balance).
3. Finally, the Copy Constructor is called when "acc3" is created using "acc2" as the initializer, since the compiler detects that an existing object is being used to initialize a new object of the same class.

In general, C++ calls the constructor that matches the way the object is being initialized: no arguments trigger the default constructor, explicit values trigger the parameterized constructor, and initialization from another object of the same class triggers the copy constructor. The order of calls simply follows the order in which the objects are declared in the program.

Question 2: Inventory Product Management System

Code:

```
#include <iostream>
#include <cstring>
using namespace std;

class Product {
private:
char* name;
int productID;
int stockQuantity;
float price;

public:
// Parameterized Constructor
Product(int id, const char* pName, int stock, float p) {
productID = id;
name = new char[strlen(pName) + 1];
strcpy(name, pName);
stockQuantity = stock;
price = p;
cout << "[Parameterized Constructor] Product created -> ID: "
<< productID << ", Name: " << name << endl;
}

// Copy Constructor
Product(const Product& other) {
productID = other.productID;
```

```
stockQuantity = other.stockQuantity;
price = other.price;
name = new char[strlen(other.name) + 1];
strcpy(name, other.name);
cout << "[Copy Constructor] Product copied -> ID: "
<< productID << ", Name: " << name << endl;
}

// Destructor
~Product() {
cout << "[Destructor] Releasing product -> ID: "
<< productID << ", Name: " << name << endl;
delete[] name;
}

// Overloaded + operator to add stock quantities
Product operator+(const Product& other) {
int combinedStock = this->stockQuantity + other.stockQuantity;
Product temp(999, "CombinedStock", combinedStock, 0.0f);
return temp;
}

// Display product details
void display() const {
cout << "Product ID: " << productID
<< " | Name: " << name
<< " | Stock: " << stockQuantity
<< " | Price: " << price << endl;
}
};

int main() {
cout << "----- Creating Product 1 using Parameterized
Constructor -----" << endl;
Product p1(101, "Laptop", 50, 55000.0f);

cout << "\n----- Creating Product 2 using Parameterized
Constructor -----" << endl;
Product p2(102, "Mouse", 150, 500.0f);

cout << "\n----- Creating Product 3 using Copy Constructor (copy
of p1) -----" << endl;
Product p3 = p1;
```

```
cout << "\n----- Adding Stock of Product 1 and Product 2 using
Overloaded + Operator -----" << endl;
Product p4 = p1 + p2;

cout << "\n----- Displaying Product Details -----" << endl;
p1.display();
p2.display();
p3.display();
p4.display();

cout << "\n----- End of main(): Destructors will now be called
in reverse order -----" << endl;
return 0;
}
```

Output:

```
----- Creating Product 1 using Parameterized Constructor -----
[Parameterized Constructor] Product created -> ID: 101, Name: Laptop

----- Creating Product 2 using Parameterized Constructor -----
[Parameterized Constructor] Product created -> ID: 102, Name: Mouse

----- Creating Product 3 using Copy Constructor (copy of p1) -----
[Copy Constructor] Product copied -> ID: 101, Name: Laptop
```

```
----- Adding Stock of Product 1 and Product 2 using Overloaded + Operator -----
[Parameterized Constructor] Product created -> ID: 999, Name: CombinedStock

----- Displaying Product Details -----
Product ID: 101 | Name: Laptop | Stock: 50 | Price: 55000
Product ID: 102 | Name: Mouse | Stock: 150 | Price: 500
Product ID: 101 | Name: Laptop | Stock: 50 | Price: 55000
Product ID: 999 | Name: CombinedStock | Stock: 200 | Price: 0
```

```
----- End of main(): Destructors will now be called in reverse order -----
[Destructor] Releasing product -> ID: 999, Name: CombinedStock
[Destructor] Releasing product -> ID: 101, Name: Laptop
[Destructor] Releasing product -> ID: 102, Name: Mouse
[Destructor] Releasing product -> ID: 101, Name: Laptop
```

Task 10: Trace and Explanation of the Object Lifecycle

1. Initialization Phase (Parameterized Constructors)

- **Action:** `Product prod1("Laptop", 101, 50);` and `Product prod2("Mouse", 102, 150);` are declared.
- **Mechanism:** The **Parameterized Constructor** is invoked for each object. It allocates memory on the stack and initializes the `name`, `productId`, and `stockQuantity` members with the values provided.
- **Output Trace:** `[Constructor] Created product: Laptop...` and `[Constructor] Created product: Mouse...`

2. Duplication Phase (Copy Constructor)

- **Action:** `Product prod3 = prod1;`
- **Mechanism:** Since we are initializing a new object (`prod3`) from an existing object (`prod1`), the **Copy Constructor** is called. It copies the values from `prod1` into `prod3` (in my code, I added " (Copy)" to the name to visually prove the copy constructor was used).
- **Output Trace:** `[Copy Constructor] Duplicated product: Laptop (Copy)`

3. Operation Phase (Operator Overloading)

- **Action:** `Product prod4 = prod1 + prod2;`
- **Mechanism:** This is a two-step process:
    - **Step A:** The `operator+` function is called. It calculates the sum of the stock quantities.
    - **Step B:** A temporary object is created within the operator function to hold the result. This temporary object is then passed to the **Copy Constructor** to initialize `prod4`.
- **Output Trace:** `[Operator +] Adding stock...`

4. Destruction Phase (Destructors)

- **Action:** The `main()` function reaches `return 0;` and the scope ends.
- **Mechanism:** C++ follows the **LIFO (Last-In, First-Out)** principle for stack-allocated objects. The objects are destroyed in the exact reverse order of their creation.
- **Sequence of Destruction:**
    1. `prod4` is destroyed (it was the last one created).
    2. `prod3` is destroyed.
    3. `prod2` is destroyed.
    4. `prod1` is destroyed (it was the first one created).
- **Output Trace:** `[Destructor] Destroying product: ...` (repeated for all four objects).

Question 3: Shopping Cart Management System

Code:

```
#include <iostream>
#include <string>

using namespace std;

// Task 1: Design the Product class
class Product {
private:
string* name;
int quantity;

public:
// Task 2: Implement a parameterized constructor
Product(string n, int q) {
name = new string(n); // Dynamically allocating memory for the
name
quantity = q;
cout << "[Constructor] Created product: " << *name << " with
quantity " << quantity << endl;
}

// Task 3: Implement a copy constructor to duplicate a product
Product(const Product& other) {
name = new string(*other.name); // Deep copy to ensure
independent memory
quantity = other.quantity;
cout << "[Copy Constructor] Duplicated product: " << *name <<
endl;
}

// Task 4: Implement a destructor to release resources
~Product() {
cout << "[Destructor] Releasing memory for product: " << *name
<< endl;
delete name; // Releasing dynamically allocated memory
}

// Task 5: Overload the + operator to add stock quantities
Product operator+(const Product& other) {
string newName = *this->name + " & " + other.name;
int newQty = this->quantity + other.quantity;
return Product(newName, newQty);
```

```
}

// Method to display product details
void display() const {
cout << "Product Name: " << *name << " | Quantity: " << quantity
<< endl;
}
};

int main() {
cout << "--- Starting Shopping Cart System ---" << endl << endl;

// Task 6: Create two product objects using parameterized
constructor
Product p1("Laptop", 10);
Product p2("Mouse", 50);

cout << endl;

// Task 7: Create another product using the copy constructor
Product p3 = p1;

cout << endl;

// Task 8: Add two products using the overloaded + operator
// This creates a temporary object via the + operator
Product p4 = p1 + p2;

cout << endl;

// Task 9: Display product details and resulting stock quantity
cout << "--- Displaying Details ---" << endl;
p1.display();
p2.display();
p3.display();
p4.display();

cout << endl << "--- End of Main (Destructors will trigger now)
---" << endl;

return 0;
}
```

Output:

```
--- Starting Shopping Cart System ---

[Constructor] Created product: Laptop with quantity 10
[Constructor] Created product: Mouse with quantity 50

[Copy Constructor] Duplicated product: Laptop

[Constructor] Created product: Laptop & Mouse with quantity 60

--- Displaying Details ---
Product Name: Laptop | Quantity: 10
Product Name: Mouse | Quantity: 50
Product Name: Laptop | Quantity: 10
Product Name: Laptop & Mouse | Quantity: 60


--- End of Main (Destructors will trigger now) ---
[Destructor] Releasing memory for product: Laptop & Mouse
[Destructor] Releasing memory for product: Laptop
[Destructor] Releasing memory for product: Mouse
[Destructor] Releasing memory for product: Laptop
```

Task 10: Trace and explanation of Constructor/Destructor sequence

When you run this program, the sequence is as follows:

1. **`p1` Constructor**: Called when "Laptop" is initialized.
2. **`p2` Constructor**: Called when "Mouse" is initialized.
3. **`p3` Copy Constructor**: Called because `p3` is initialized using `p1`. It performs a "deep copy" of the string.
4. **`p1 + p2` Operation**:
    - The `+` operator creates a **temporary Product object** to hold the result of the addition.
    - The **Parameterized Constructor** is called for this temporary object.
5. **`p4` Copy Constructor**: Since the result of `p1 + p2` is being assigned to `p4`, a copy constructor is called to transfer the data from the temporary object to `p4`.
6. **Destructors**: As the `main()` function ends, objects are destroyed in the **reverse order** of their creation:
    - `p4` is destroyed.
    - The temporary object from the addition is destroyed.

- `p3` is destroyed.
- `p2` is destroyed.
- `p1` is destroyed.

Task 11: Memory Leak Explanation

If dynamically allocated memory (like the `name` pointer in this program) is not released using the `delete` keyword in the destructor, a **Memory Leak** occurs. This means the memory remains "occupied" in the eyes of the Operating System even though the program no longer has a pointer to access it. If a program runs for a long time and continuously leaks memory, it will eventually consume all available system RAM, leading to system slowdowns or the application crashing.